StoreHub — Progress Report & AWS Deployment Roadmap

Repo: `Omar-Shaker-Elbana/Store_Hub` | **Reviewed:** July 19, 2026 | **Stack:** Django 6.0, SQLite (dev)

This is a direct code-level audit — not just a README read-through. Every claim below was verified against the actual source (models, views, urls, settings, CI config).

---

1. Progress Snapshot

Module	Status	Real completion	Notes
`landing_page`	🟡 Minimal	~90% (of its small scope)	Just renders a static home page. No dynamic content.
`users` (auth/profile)	🟡 Functional, flawed	~65%	Registration/login/profile work, but the custom email backend is dead code (see §2).
`merchant_interface`	🟡 Functional, buggy	~60%	Store + membership CRUD works; several redirects are broken (§2). Analytics is an empty placeholder.
`products`	🟡 Functional, buggy	~55%	CRUD works; one data-model bug undermines the review feature (§2); two views will crash on update.
`orders`	🔴 Not started	0%	Empty Django scaffold — no models, no views, no urls registered.
`notifications`	🔴 Not started	0%	Empty Django scaffold.
`shopper_interface`	🔴 Not started	0%	Empty Django scaffold — cart/checkout doesn't exist yet.
Tests	🔴 None	0%	Every `tests.py` in every app is the untouched Django stub.
CI (`django.yml`)	🔴 Broken right now	—	`requirements.txt` is saved as UTF-16, which breaks `pip install -r`

Module	Status	Real completion	Notes
			`requirements.txt` in CI. This pipeline is very likely red on every push.
Production infra	🔴 Not started	0%	`DEBUG=True`, secret key committed, SQLite, no env config, no static/media storage plan, nothing AWS-related exists yet.

Honest overall read: ~35–40% of the vision in your README is actually built. The auth/store/product core is real and mostly works. Commerce (cart → order → notify → pay) is entirely unbuilt, there's zero automated test coverage, and the project isn't deployment-ready in its current state — not because the idea is behind, but because infra/hardening was deferred, which is normal at this stage.

---

2. Confirmed Bugs (verified in code, not guesses)

These are worth fixing before anyone builds on top of them:

1. **CI is almost certainly broken.** `requirements.txt` is UTF-16LE encoded (likely from `pip freeze` on Windows PowerShell without `-Encoding utf8`). `pip` expects UTF-8/ASCII — this will raise a decode error in your GitHub Actions workflow. Regenerate it as plain UTF-8.
2. **Broken redirects (404s) in `merchant_interface/views.py`.** `create_store`, `add_members`, and `edit_store` redirect to hardcoded paths like `/add_members/{id}` — but the actual route (per `urls.py`) is `/merchant/add_members/<id>/`. Every one of these redirects will 404 for a real user. Fix: use `redirect('add_members', store_id=store.id)` (named URL reversal) everywhere instead of hand-typed paths.
3. **`Review.product` is a `OneToOneField`, not a `ForeignKey`** (`products/models.py`). This means **only one review can ever exist per product, across all users** — not "one review per user per product" as the README states. The `UniqueConstraint(user, product)` right below it is dead code because the `OneToOneField` already caps it at one row total. This is a real bug in the core review feature, not a style nit.
4. **Two views will crash on execution.** In `products/views.py`, `Update_Product` and `Update_Spec` both call `messages.success("...")` without the required `request` argument (should be `messages.success(request, "...")`). Both will raise a `TypeError` the moment a user successfully updates a product or a spec.
5. **`Suggest_Category` bypasses its own moderation model.** You built a `SuggestedCategory` model specifically for admin review, but the view writes straight

into the live `Category` table instead, skipping the approval step entirely. Either wire it to `SuggestedCategory` or drop the unused model.

6. **`AUTHENTICATION_BACKENDS` is commented out in `settings.py`.** Your custom `EmailBackend` (`users/backends.py`) is never actually used — login currently works only because `username` is silently set equal to `email` at registration. It functions today, but it's an accident, not the design you documented. Decide: keep the username=email trick and delete the unused backend, or activate the backend properly.
7. **Two dead views with no URL routes.** `my_store` and `my_analytics` exist in `merchant_interface/views.py` but aren't registered in `urls.py` — currently unreachable.
8. **Inconsistent/invalid model defaults** in `merchant_interface/models.py`: `Membership.wage_type` defaults to `'helper'`, but valid choices are only `'Salary'`/`'Percentage'` — that default is not a valid option at all.
9. **Repo clutter that should be removed before onboarding teammates:** `new.py` (a fully commented-out scratch file at the project root) and `package-lock.json` (an empty, unused npm artifact in a pure-Django project).

None of this is catastrophic — it's exactly what you'd expect from a project built in bursts and then paused. But #1–#4 will actively break things for real users, so they're the first fixes, before any new feature work.

---

3. Infrastructure / Security Gaps (relevant since you're targeting AWS)

- `SECRET_KEY` is hardcoded and committed to git — must be rotated and moved to an environment variable before any deploy.
- `DEBUG = True` is hardcoded, not env-controlled — must be `False` in production, or you'll leak stack traces publicly.
- `ALLOWED_HOSTS = []` — will hard-reject all requests once `DEBUG=False`. Needs your domain/ELB DNS added via env var.
- No `django-environ` / `python-dotenv` / `.env` handling exists yet — the README documents this pattern but it isn't implemented.
- SQLite is the active database — fine for dev, not for a multi-instance AWS deployment (no concurrent write safety, can't live on ephemeral compute).
- No `django-storages`/`boto3` in `requirements.txt` — needed before media (product/store/profile images) can live on S3 instead of local disk, which is required if you run on more than one instance or use ECS/Elastic Beanstalk with ephemeral storage.

- No `gunicorn`/`whitenoise`/production WSGI server in dependencies — `manage.py runserver` is dev-only.
- No logging, health-check endpoint, or error-monitoring hook (e.g. Sentry) yet — you'll want these before production traffic.

---

4. Roadmap & Milestones

Structured in phases so backend and frontend can work in parallel without blocking each other. Suggested owners based on your README's team roles (Backend: you + Ziad; Frontend: Mark + Ahmed).

Phase 0 — Stabilization (backend-heavy, ~1 week)

Goal: nothing in the current feature set is broken before you build more on top of it.

- [ ] Fix `requirements.txt` encoding (UTF-8), confirm CI goes green
- [ ] Fix the 4 confirmed bugs in §2 (redirects, `Review` model, `messages.success` crashes, category bypass)
- [ ] Decide and fix the auth backend situation (activate `EmailBackend` properly or remove it)
- [ ] Register or delete the two dead views (`my_store`, `my_analytics`)
- [ ] Remove `new.py`, `package-lock.json`
- [ ] Move `SECRET_KEY`, `DEBUG`, `ALLOWED_HOSTS` to environment variables (`django-environ`), add `.env.example`

Owner: Backend (you + Ziad)

Phase 1 — Complete the commerce core (backend + frontend, ~3–4 weeks)

Goal: a shopper can actually go from browsing to a placed order.

- [ ] `shopper_interface`: cart model + add/remove/update-quantity views, wishlist
- [ ] `orders`: `Order`, `OrderItem` models, order lifecycle (placed → confirmed → shipped → delivered), stock decrement on order, merchant-side order management views
- [ ] `notifications`: `Notification` model, triggers for invitations/order status changes, at minimum an in-app notification list (email delivery can come later)
- [ ] Frontend: cart UI, checkout flow, order history page, merchant order dashboard
- [ ] Role-based permission pass: decide explicitly whether `Helper`/`Manager`/`Owner` should have different product/order permissions (currently any member can do anything in `products` views) and enforce it consistently

Owner: Backend (models/views) + Frontend (Mark/Ahmed on templates & UX)

Phase 2 — Testing & hardening (~2 weeks, can overlap with Phase 1's tail end)

- [ ] Test suite: model constraints, auth flow, permission boundaries, order lifecycle — target meaningful coverage on `users`, `merchant_interface`, `products`, `orders` before touching payments
- [ ] Switch dev DB to Postgres locally (matches production, catches SQLite-specific bugs early)
- [ ] Add `django-storages` + S3 for media uploads
- [ ] Security pass: CSRF/session settings for production, password reset flow (currently missing), rate limiting on auth endpoints

Owner: Backend

Phase 3 — AWS Infrastructure (~2–3 weeks)

Goal: a real, repeatable production deployment — not a one-off manual setup.

- [ ] Containerize with Docker (Dockerfile + docker-compose for local parity)
- [ ] **Database:** RDS PostgreSQL (Multi-AZ optional at your scale; start single-AZ + automated backups)
- [ ] **Media/static:** S3 bucket for media, served via CloudFront; static files via WhiteNoise or S3+CloudFront
- [ ] **App hosting:** pick one — Elastic Beanstalk (fastest to stand up, good for a small team) or ECS Fargate (more control, better long-term fit if you plan to add services like a payment worker or background jobs)
- [ ] **Secrets:** AWS Secrets Manager or SSM Parameter Store for `SECRET_KEY`, DB creds — never in env files committed to git
- [ ] **Networking:** VPC with private subnets for RDS, public subnet + ALB for the app, HTTPS via ACM certificate
- [ ] **DNS:** Route 53 for your domain
- [ ] **CI/CD:** extend the existing GitHub Actions workflow to build the Docker image and deploy on merge to `main` (after Phase 0 fixes it)
- [ ] **Observability:** CloudWatch logs/alarms at minimum; Sentry for error tracking is a cheap, high-value add

Owner: Backend/infra (you, since this is your stated deploy target — good opportunity to own this end-to-end)

Phase 4 — Payments, analytics, launch (~2+ weeks)

- [ ] Payment gateway integration (Stripe or Paymob, per your README)
- [ ] Store analytics dashboard (the `my_analytics` placeholder becomes real)
- [ ] REST API layer (DRF) if you want a future mobile app or third-party integrations

- [ ] Load-test the checkout path before public launch
  Owner: Backend + Frontend joint

---

5. Suggested Immediate Next Step

Do Phase 0 first, in one focused sprint, before assigning any new feature work to the team. Right now your CI is silently broken and three of your "done" features (reviews, store editing, product/spec updates) have live bugs — onboarding teammates onto a codebase with unnoticed breakage costs more time than fixing it up front. Once Phase 0 is green, Phases 1 and 3 (commerce core + AWS infra) can run in parallel on separate branches since they don't depend on each other.